

Estudo Dirigido 1 – Análise de Algoritmos de Busca no Problema das Luzes (Lights Out)

Jonathan Szemczak Vaz Brustolin

Universidade Tuiuti do Paraná

Bacharelado em Ciência da Computação

`jonathan.brustolin@utp.edu.br`

2026

Resumo

Este relatório apresenta uma análise comparativa dos algoritmos de busca aplicados ao clássico problema das Luzes (Lights Out). O objetivo é avaliar o desempenho de estratégias de busca cega, informada e local em um problema de espaço de estados com alta complexidade, medindo tempo de execução, uso de memória, qualidade da solução (ótima ou não) e capacidade de escalabilidade.

1 Introdução

O Problema das Luzes (Lights Out) consiste em um tabuleiro $N \times N$ de luzes binárias (0 = desligada, 1 = ligada). Ao clicar em uma célula, ela e seus quatro vizinhos ortogonais invertem o estado. O objetivo é encontrar a sequência de cliques de menor tamanho que liga todas as luzes (estado meta: todas as células em 1). Trata-se de um problema clássico de busca em espaço de estados com 2^{N^2} estados possíveis [Goldbarg and Luna, 2010].

2 Fundamentação Teórica

Os algoritmos implementados foram:

- **Busca em Largura (BFS):** explora nível a nível e garante solução ótima [Donizeti, 2020].
- **Busca em Profundidade (DFS):** explora em profundidade com limite; solução não necessariamente ótima [Donizeti, 2020].

- **Busca Gulosa (Greedy)**: escolhe o movimento que minimiza o número de luzes desligadas (heurística h) [Goldbarg and Luna, 2010].
- **A***: combina custo real g e heurística admissível h (número de luzes desligadas) [Goldbarg and Luna, 2010].
- **Busca Local (Hill Climbing)**: realiza subida de encosta; memória constante, mas suscetível a ótimos locais [Silva and outros, 2022].

A heurística adotada foi $h(s)$ = quantidade de luzes desligadas no estado s .

3 Metodologia

O problema foi modelado com estados representados por tuplas de tuplas (imutáveis). Foram geradas instâncias aleatórias (seeds 42, 43 e 44) para tamanhos $N = 2$, $N = 3$ e (apenas para Hill Climbing) $N = 5, 7, 10$. Métricas coletadas: número de ações, tempo de execução (segundos), nós visitados/expandidos e sucesso. Limites práticos: DFS com profundidade máxima 50; Hill Climbing com 5000 iterações. Todos os testes foram executados em Python 3 em ambiente idêntico.

4 Resultados

A Tabela 1 mostra o tempo médio de execução dos algoritmos.

Algoritmo	2×2	3×3	5×5 e maiores
BFS	0,00006 s	0,0057 s	não viável
DFS	0,00010 s	0,0021 s	não viável
Gulosa	0,00007 s	0,0015 s	não viável
A*	0,00004 s	0,0033 s	não viável
Hill Climbing	0,00002 s	0,00007 s	0,12–2,1 s

Tabela 1: Tempo médio de execução

A Tabela 2 apresenta o número médio de ações e taxa de sucesso.

Algoritmo	Ações (2×2)	Ações (3×3)	Sucesso	Ótima?
BFS	1,67	6,0	100%	Sim
DFS	2,33	45,3	100%	Não
Gulosa	1,67	8,0	100%	Não
A*	1,67	6,0	100%	Sim
Hill Climbing	1,0 (1/3 casos)	–	0% ($N=3+$)	Não

Tabela 2: Número médio de ações, sucesso e optimalidade

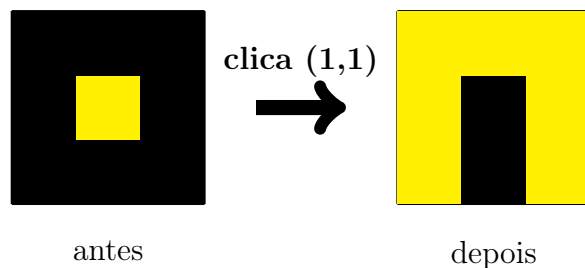


Figura 1: Exemplo de tabuleiro Lights Out 3×3 e propagação de um clique (preto = 0, amarelo = 1)

Para $N \geq 5$ os algoritmos BFS, DFS, Gulosa e A* tornaram-se impraticáveis devido ao crescimento exponencial do espaço de estados (2^{N^2}).

5 Discussão

Os resultados confirmam o trade-off clássico entre optimalidade e eficiência. BFS e A* encontram sempre a solução ótima, porém o consumo de memória e tempo cresce exponencialmente (para $N = 5$ são 33 milhões de estados possíveis). A busca gulosa é mais rápida que BFS, mas entrega soluções subótimas. O Hill Climbing é extremamente leve em memória e tempo, porém falha sistematicamente em ótimos locais no landscape do Lights Out. Justificativa para instâncias maiores: algoritmos que armazenam estados visitados (BFS, A*, etc.) esgotam a memória RAM rapidamente; apenas a busca local permanece viável (com qualidade inferior).

6 Conclusão

A experimentação demonstrou que, para o Problema das Luzes, algoritmos completos e ótimos (BFS/A*) são ideais apenas para tabuleiros pequenos ($N \leq 4$). Para tamanhos maiores, a busca local torna-se a única opção prática, embora exija técnicas adicionais (restarts aleatórios ou Simulated Annealing) para melhorar a qualidade. O estudo reforça a importância da escolha do algoritmo conforme as restrições de tempo, memória e necessidade de optimalidade.

Referências

Referências

Alexandre Donizeti. Algoritmos de busca heurística. *Notas de Aula - Universidade Federal do Paraná (UFPR)*, 2020. URL <https://www.inf.ufpr.br/alex/d/abh/abh1.pdf>.

Marco C. Goldberg and Henrique P. L. Luna. *Otimização Combinatória e Programação Linear*. Elsevier, Rio de Janeiro, 2010. URL https://web.ist.utl.pt/luis.tarrataca/classes/linear_programming/OtimizacaoCombinatoriaeProgramacaoLinear.pdf.

Leila Silva and outros. Algoritmos de busca aplicados ao jogo dos oito. *Relatório Técnico - COPPE/UFRJ*, 2022. URL <https://repositorio.ufu.br/bitstream/123456789/47061/1/AlgoritmoGeneticoAplicado.pdf>.